

# Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis

Jarin Tasnim

Department of CSE, BUET

2005083@ugrad.cse.buet.ac.bd

Abdullah Faiyaz

Department of CSE, BUET

2005065@ugrad.cse.buet.ac.bd

Mohammad Sadnam Faiyaz

Department of CSE, BUET

2005111@ugrad.cse.buet.ac.bd

Md. Mostofa Akbar

Department of CSE, BUET

mostofa@cse.buet.ac.bd

**Abstract**—Android malware continues to grow in volume and sophistication, yet most high-accuracy detectors rely on cloud backends or heavyweight analysis that is unsuitable for offline, privacy-preserving screening on commodity phones. We present **VigiDroid**, a practical on-device static malware pre-screener that fuses multiple literature-derived lightweight models through ONNX Runtime.

Our workflow has two stages: (i) offline training on a large benign/malware APK corpus (~500 GB, including AndroZoo-sourced samples) with temporal train/test splits, PyTorch training, and ONNX export with train/serve parity checks; and (ii) on-device inference in an Android app that parses APKs locally, runs tiered model execution (Quick / Balanced / Full), and fuses scores via a configurable ensemble policy. **VigiDroid** scans all `classes*.dex` files for structural models, records per-stage wall time, process-level CPU, multi-dimensional memory, and battery deltas, and exports structured JSON metrics for thesis evaluation.

**Index Terms**—Android malware detection, static analysis, on-device inference, ONNX, ensemble learning, mobile security

## I. INTRODUCTION

Mobile malware detection has seen strong results with various static and dynamic analysis methods over the years. Yet surprisingly few efforts have focused on what actually matters in real life: making these techniques work efficiently on everyday phones with limited battery, CPU, and memory. In this paper, we present **VigiDroid**, a fully offline static analysis framework that runs entirely on the device itself. We experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and even raw APK byte sequences) paired with lightweight models such as XGBoost, multilayer perceptrons, and 1D CNNs. All models are deployed using ONNX Runtime and evaluated directly on Android devices. In addition to detection accuracy and F1-score, we profile feature extraction, vectorization and inference time, CPU utilization, and memory consumption. Preliminary experiments show that [mention experimental best result + resource usage tradeoff].

## II. BACKGROUND AND RELATED WORKS

Android malware detection has been studied extensively using both static and dynamic analysis. Like permission-based linear models [2], broadcast-receiver mining [4], permission extraction frameworks (MLDP) [8], byte-level 1-D CNNs on raw APK tails [1], and structural DEX/manifest fusion architectures inspired by MSFDroid-style [11] multi-branch

designs. Early efforts like SeqMobile [5] showed that on-device detection is possible by using RNN-based models on permissions, intents, and API call sequences. SAMADroid [6] took a hybrid approach, combining lightweight on-device checks with cloud-based dynamic analysis. However, its reliance on certain system privileges has become problematic as modern Android versions have tightened security restrictions. DL-Droid [7] ran dynamic analysis directly on real devices, but it required installing and fully executing each app which make it impractical for everyday scanning. Overall, these works highlight that truly practical, resource-efficient on-device malware detection is still an open challenge, especially when balancing strong detection performance against tight mobile resource constraints.

## III. PROPOSED METHODOLOGY

### A. Model Development

The offline half of our pipeline follows a consistent eight-phase structure (P0–P8) applied independently to each detector: environment setup and configuration, APK corpus indexing, feature extraction, dataset loader construction, model definition, training, held-out test evaluation, ONNX export, and finally a parity check confirming that the exported model reproduces PyTorch probabilities within  $10^{-4}$  on a set of representative samples. Enforcing this parity gate before any Android integration ensures that accuracy figures reported from the PC are meaningful on-device.

Our training corpus comprises roughly 500 GB of APKs drawn from AndroZoo and related sources, organized by year and label. We adopt a temporal split throughout: models are trained on 2020–2021 samples and evaluated on 2022–2023 samples, which mirrors the realistic scenario where a deployed detector faces APKs newer than those it was trained on. Validation folds are always drawn from the training years to prevent future leakage.

We implement nine model families spanning a range of feature domains and computational costs, grouped into three inference tiers. At the **LIGHT** tier, four fast models handle manifest-level and raw-byte signals: **ByteCNN** [1] applies a 1-D convolutional network to the last 1024 bytes of the APK; **LinRegDroid** [2] feeds a normalized manifest permission vector into a linear classifier; **MLDP-pruned** [8] uses a compact MLP over a permission subset selected by the MLDP

framework; and a **Broadcast+MLDP hybrid** [4], [8] combines static receiver-action features with MLDP permissions through a small fusion network. At the **MID** tier, we add structural DEX signals: **MLP(H)** (BM1), a 104-dimensional normalized DEX header classifier inspired by MSFDroid; **Pattern A**, which performs early fusion of the DEX header with a manifest bag-of-words over a roughly 4,380-entry lexicon; **Pattern B**, a dual-branch variant that fuses the same two signals via late fusion; and two **MLDP+DEX header cascade** configurations (modes A and B) that route samples through a permission gate before invoking the header branch. Finally, at the **HEAVY** tier, a pre-integrated **XGBoost** model aggregates manifest tokens across all `classes*.dex` files into a 2,500-dimensional OR vector for gradient-boosted inference.

Where an original paper’s architecture was not directly portable to ONNX or mobile inference, we adopted lightweight substitutes—replacing SVM/tree classifiers with small MLPs or linear heads—while preserving the published feature semantics. Each model is exported as a self-contained bundle containing `model.onnx`, an `export_manifest.json` describing input shapes and domains, threshold files, feature vocabulary or normalization assets, and a small set of parity samples. These bundles are copied directly into VigiDroid’s asset directory, making the PC-to-phone handoff a straightforward file transfer.

### B. On-Device System: VigiDroid

VigiDroid is an offline-first Android application that runs all inference locally using ONNX Runtime, without sending any APK data to a remote server. When a user initiates a scan, they first choose a scan depth—**Quick**, **Balanced**, or **Full**—which controls which model tiers are active. Quick mode runs only the **LIGHT** tier for a fast initial screen; Balanced adds the **MID** tier for better accuracy at moderate cost; Full enables all three tiers including the **HEAVY** XGBoost model.

For each APK found in the device’s `Download/` folder, `ScanService` opens a metered session and runs the enabled models in sequence. Java feature extractors inside the app replicate the Python extraction logic from P2 exactly, using the vocabulary and normalization assets shipped in the export bundle—this replication is a strict requirement, since any drift between Python and Java feature semantics would silently invalidate on-device accuracy. After each model produces a score, `EnsemblePolicy` fuses the available outputs into a final verdict of benign, malware, or uncertain. Three fusion policies are supported: a legacy mode using only XGBoost and ByteCNN for backward compatibility; a default weighted-average mode that combines all models that ran, with weights derived from offline validation accuracy; and a tiered cascade mode that escalates from **LIGHT** to **MID** or **HEAVY** only when the lighter models are inconclusive.

Throughout each scan, VigiDroid records a detailed operational trace. Per-stage measurements include wall-clock time, whole-process CPU consumption, per-thread CPU time for the scan worker, native and Java heap deltas, proportional set size (PSS) changes, and battery percentage change over the

session. These metrics are appended to a structured JSON log (`all_scan_metrics.json`) that accumulates across sessions and can be pulled from the device for post-hoc analysis.

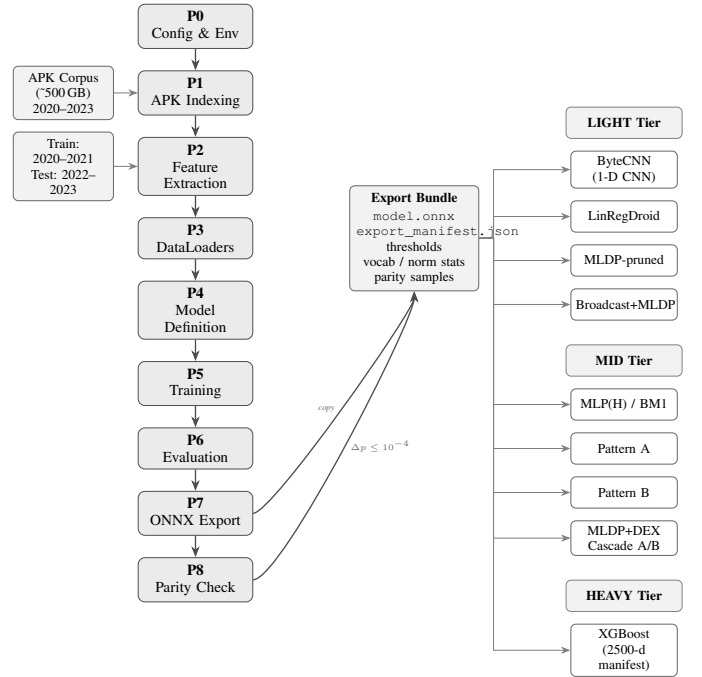


Fig. 1. Offline VigiDroid model-development pipeline. The PC-side workflow indexes APKs, extracts features, trains and exports ONNX bundles, and organizes the exported detectors into **LIGHT**, **MID**, and **HEAVY** inference tiers.

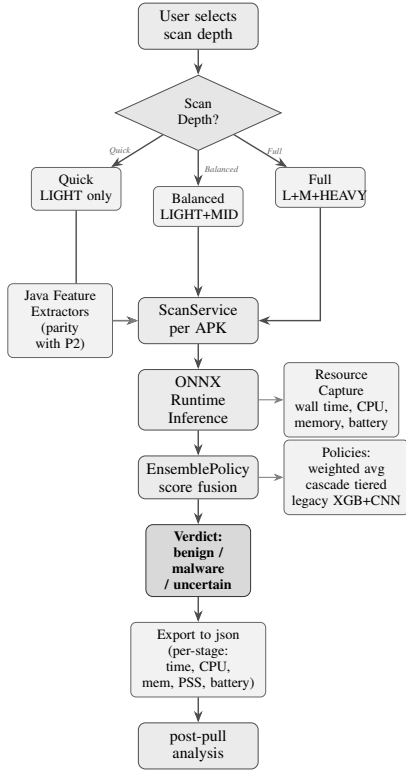


Fig. 2. VigiDroid on-device application flow. The app selects a scan depth, runs the corresponding model tiers through local feature extraction and ONNX Runtime inference, fuses model scores, reports a verdict, and writes resource metrics for post-device analysis.

#### IV. EXPERIMENTS

We evaluate each model on two axes: **offline quality** (accuracy, F1, ROC-AUC on temporal holdout APKs) and **on-device feasibility** (mean CPU, memory, battery, wall time per scan on a physical Android device). Tables I and II split the planned comparison into offline quality and on-device feasibility metrics; values below are placeholders pending completed device campaigns and unified offline exports. Fig. 3 illustrates (placeholder).

TABLE I  
PLANNED OFFLINE QUALITY COMPARISON OF THESIS MODELS (VALUES TBD). METRICS ARE COMPUTED ON THE PC TEMPORAL TEST SPLIT.

| Model            | Features             | Acc.  | F1    | ROC-AUC |
|------------------|----------------------|-------|-------|---------|
| 1-D CNN          | Last 1024 APK bytes  | —     | —     | —       |
| LinRegDroid      | Manifest permissions | —     | —     | —       |
| MLDP-pruned      | MLDP permissions     | —     | —     | —       |
| Broadcast+MLDP   | Perms.+ receivers    | —     | —     | —       |
| MLP(H) / BM1     | DEX header           | 0.966 | 0.944 | 0.963   |
| Pattern A        | Header+ manifest BoW | —     | —     | —       |
| Pattern B        | Dual-branch fusion   | —     | —     | —       |
| MLDP+DEX cascade | MLDP    header       | —     | —     | —       |
| XGBoost          | Multi-DEX manifest   | —     | —     | —       |

TABLE II  
PLANNED ON-DEVICE FEASIBILITY COMPARISON OF THESIS MODELS (VALUES TBD). RESOURCE COLUMNS ARE DERIVED FROM ALL\_SCAN\_METRICS.JSON.

| Model            | CPU | Mem. | Batt. | Time | Feasible |
|------------------|-----|------|-------|------|----------|
| 1-D CNN          | —   | —    | —     | —    | TBD      |
| LinRegDroid      | —   | —    | —     | —    | TBD      |
| MLDP-pruned      | —   | —    | —     | —    | TBD      |
| Broadcast+MLDP   | —   | —    | —     | —    | TBD      |
| MLP(H) / BM1     | —   | —    | —     | —    | TBD      |
| Pattern A        | —   | —    | —     | —    | TBD      |
| Pattern B        | —   | —    | —     | —    | TBD      |
| MLDP+DEX cascade | —   | —    | —     | —    | TBD      |
| XGBoost          | —   | —    | —     | —    | TBD      |

**[Figure 1 — Placeholder]**  
Inference time vs apk size:  
UI → ScanService → tiered ONNX models → ensemble → metrics JSON

Fig. 3. inference time vs apk size

**[Figure 2 — Placeholder]**  
Planned: accuracy vs. mean scan time (or Pareto plot) across models and scan levels

Fig. 4. Planned trade-off figure: detection quality versus on-device cost across models and Quick/Balanced/Full scan depths. To be generated from offline + device metrics.

**Protocol (planned).** Labeled eval APKs (eval\_XXXX\_benign/malware.apk) on device; metrics pulled via ADB; offline labels from corpus year splits.

#### V. CONCLUSION

We described VigiDroid, a thesis framework for practical on-device Android malware pre-screening that unifies literature-based static detectors through a disciplined train-export-deploy pipeline and tiered ONNX orchestration on Android. By logging multi-dimensional resource metrics alongside fused verdicts, the system supports evidence-based selection of models that balance accuracy and mobile cost. Final quantitative results, ensemble calibration, and user-study implications will be reported once device experiments and figure generation are complete.

#### ACKNOWLEDGMENT

The authors thank their supervisor and collaborators at the Department of Computer Science and Engineering, BUET, for guidance throughout this thesis.

## REFERENCES

- [1] C. Hasegawa and H. Iyatomi, "One-dimensional convolutional neural networks for Android malware detection," in *Proc. ICCE*, 2020.
- [2] A. Narayanan et al., "LinRegDroid: Detection of Android malware using multiple linear regression," *Computers & Security*, 2018.
- [3] Tao Peng et al., "A Lightweight Multi-Source Fast Android Malware Detection Model," *Appl. Sci.*, 2022.
- [4] M. Mohsen and A. Ismail, "Detecting Android malwares by mining statically registered broadcast receivers," *Int. J. Advanced Computer Science and Applications*, 2017.
- [5] Ruitao Feng et al., "SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices" *25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, 2020.
- [6] Saba Arshad et al., "SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System" *IEEE Access*, 2018.
- [7] Mohammed K. Alzaylaee et al., "DL-Droid: Deep learning based android malware detection using real devices" *IEEE Access*, 2020.
- [8] S. Wang et al., "Permission extraction framework for Android malware detection," *IEEE Access*, 2019.
- [9] Y. Li et al., "MSFDroid: Multi-stage fusion for Android malware detection," *IEEE Trans. Information Forensics and Security*, 2022.
- [10] Bragança, H., Kreutz, D., Rocha, V. et al. , "MH-1M: A 1.34 Million-Sample Multi-Feature Android Malware Dataset with Rich Metadata," *Sci Data* 13, 153 . <https://doi.org/10.1038/s41597-025-06469-5>, 2026.
- [11] Tao Peng et al., "A Lightweight Multi-Source Fast Android Malware Detection Model," *Appl. Sci.*, 2022.
- [12] D. Arp et al., "Drebin: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, 2014.
- [13] ONNX Runtime documentation, <https://onnxruntime.ai/>, accessed 2026.